Verilog Introduction

Verilog HDL is one of the two most common Hardware Description Languages (HDL) used by integrated circuit (IC) designers.

The other one is VHDL. HDL's allows the design to be simulated earlier in the design cycle in order to correct errors or experiment with different architectures. Designs described in HDL are technology-independent, easy to design and debug, and are usually more readable than schematics, particularly for large circuits.

Verilog Lexical Tokens:

Lexical Tokens

Verilog source text files consists of the following lexical tokens:

White Space

White spaces separate words and can contain spaces, tabs, new-lines and form feeds. Thus a statement can extend over multiple lines without special continuation characters.

Comments

Comments can be specified in two ways:

- Begin the comment with double slashes (//). All text between these characters and the end of the line will be ignored by the Verilog compiler
- Enclose comments between the characters /* and */. Using this method allows continue comments on more than one line. This is good for "commenting out" many lines code, or for very brief in-line comments.

Identifiers

Identifiers are user-defined words for variables, function names, module names, block names and instance names. Identifiers begin with a letter or underscore (Not with a number or $) and can include any number of letters, digits and underscores.

Identifiers in Verilog are case-sensitive.

Syntax

allowed symbols : ABCDE . . . abcdef. . . 1234567890 _$

not allowed: anything else especially - &#@

Example of Valid identifiers

comparator_input

Compout$p

_datax64

Example of Invalid identifiers

&comparator1

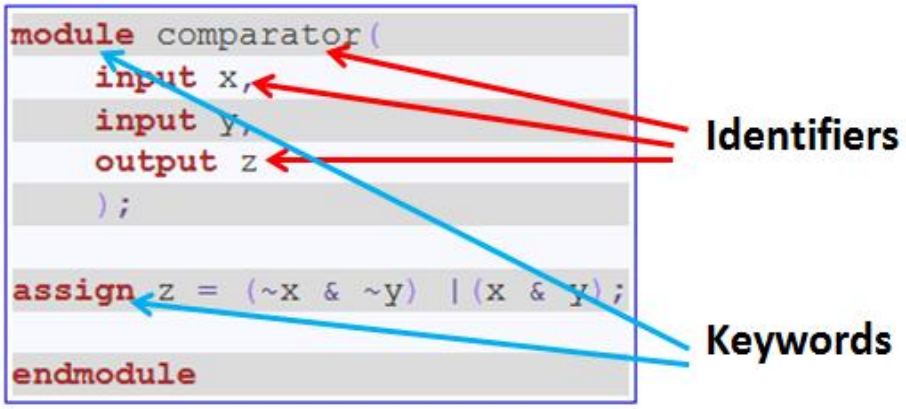
#compoutp

$_datax64

45data

Verilog Keywords

These are words that have special meaning in Verilog.
Some examples are assign, case, while, wire, reg, and, or, nand, and module.
They should not be used as identifiers.



Case Sensitivity

Verilog is case sensitive. There are couple of things you need to keep in mind

1. All the verilog keywords are in lower case.
2. If you use an identifier that starts with upper case, it will be different from the one in lower case

Example

```
wire  // A verilog keyword
WIRE  // This will not be a verilog keyword
```

Verilog: Modules

Module Declaration

A module is the principal design entity in Verilog.

The module definition starts with the keyword module followed by the module name which is an identifier to identify the name of the module.
The module contains a list of the input and output port and enclosed by round bracket parenthesis and followed by ";".
The module definition is succeeded by a list of codes describing the behaviour of the module.
Finally the module definition ends with the keyword endmodule.

Ports:
Modules are connected by Ports. A port can be an input, an output or an inout.
Syntax

```
modulemodule_name (port_list);
  input [msb:lsb] input_port_list;
  output [msb:lsb] output_port_list;
  inout [msb:lsb] inout_port_list;
        ... statements ...
endmodule
```

Summing Module and Ports
Typically a module and a port is declared as in the example below.

Example

```
module comparator(
input x,
input y,
output z
);
 assign z =(~x &~y)|(x & y);
 endmodule
```

or

```
module comparator(   x,  y,  z );
input x;
input y;
output z;
assign z =(~x &~y)|(x & y);
endmodule
```

The first line of a module declaration specifies the name and port list (arguments). The next few lines specifies the i/o type (input, output or inout,) and width of each port. The default port width is 1 bit.

There are certain rules that must be followed when connecting and input , output and the inout ports

Input Ports Internally the input ports must always be of type net. Externally the inputs can connect to a variable of type reg or net.
Outputs Internally output port can be net or reg. Externally the outputs must connect to a variable of type net.
Inouts: The Inout port must always be of type net internally or externally.

Value Set

Verilog consists of only four basic values. Almost all Verilog data types store all these values:

0 (logic zero, or false condition)
1 (logic one, or true condition)
x (unknown logic value) x and z have limited use for synthesis.
z (high impedance state)

Representing Numbers:

- Representation: **<size>'<base><number>**
  - size => number of **BITS** (regardless of base used)
  - base => base the given number is specified in
  - number => the actual value in the given base
- **Size** : Specified in decimal only and represents number of bits in number,Size defaults to at least 32
- Can use different bases
  - Decimal (d or D) – default if no base specified!
  - Hex (h or H)
  - Octal (o or O)
  - Binary (b or B)

Example

Number	Decimal Equivalent	Actual Binary
4'd3	3	0011
3'hA	10	010
8'o26	22	00010110
5'b111	7	00111
8'bx1101	-	xxxx1101
'o7	7	00000……111(32 bits)
10	10	????

*Numbers with MSB of **x** or **z** extended with that value*

Built-in Primitives

Verilog Provides in-built primitives for basic gate and switch level modeling. Any circuit can be modeled by using continuous assignment of gate and switch level primitives.

and, nand, or, nor, xor, xnorgates are the built-in primitives

These gates have one output and one or more inputs. The first port on the port list is output port.

and	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

or	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

xor	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

nand	0	1	x	z
0	1	1	1	1
1	1	0	x	x
x	1	x	x	x
z	1	x	x	x

nor	0	1	x	z
0	1	0	x	x
1	0	0	0	0
x	x	0	x	x
z	x	0	x	x

xnor	0	1	x	z
0	1	0	x	x
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

buf and not gates

These gates have one input and one or more outputs. The last port on the port list is an input port.

buffer

buf	
input	output
0	0
1	1
x	x
z	x

not	
input	output
0	1
1	0
x	x
z	x

bufif1, bufif0, notif1, notif0 Gates

These gates have three ports: the first is an output port, the second is a data port, and the third is a control port. The control port is used to set gates in high-impedance state.

bufif0		control input			
		0	1	x	z
data input	0	0	z	L	L
	1	1	z	H	H
	x	x	z	x	x
	z	x	z	x	x

bufif1		control input			
		0	1	x	z
data input	0	z	0	L	L
	1	z	1	H	H
	x	z	x	x	x
	z	z	x	x	x

The L and H symbols have a special meaning. The L symbol means that the output has 0 or z value. The H symbol means that the output has 1 or z value. Any transition to H or L is treated as a transition to x.

Verilog Data types

Net data types are used to model physical connections. They do not store values (there is only one exception - **trireg**, which stores a previously assigned value). The net data types have the value of their drivers. If a net variable has no driver, then it has a high-impedance value (z).

Nets can be declared in a net declaration statement or in a net declaration assignment.

- Nets are data types that can be used to model physical connections.
- Every declaration has a type associated with it.
- All ports declarations are implicitly declared as wire(net) type.
- Nets represent connections between hardware elements.
- Nets are declared primarily with the keyword **wire**.
- It does not store the value, therefore needs to be continuously driven. i.e. Driver is implied when a net/ wire

• Equal to*z* when unconnected

Wire and tri nets:

Both **wire** net and **tri** net are identical. Two different names are used for more readability. Preferably **wire** nets may be used if a net has only one driver. If a net has more than one driver, then **tri** net may be used instead.

Table : Truth table for wire and tri nets

	0	1	x	z
0	0	x	x	0
1	x	1	x	1
x	x	x	x	x
z	0	1	x	z

This table applies only to **wire** and **tri** nets driven by multiple drivers that have no strength declaration or their strengths are equal.

Wand and triand nets: These net types are *wired and* nets. It means that if one of their drivers is 0 then the result value will also be 0.

Table: Truth table for wand and triand nets

	0	1	x	z
0	0	0	0	0
1	0	1	x	1
x	0	x	x	x
z	0	1	x	z

This table applies only to **wand** and **triand** nets driven by multiple drivers that have no strength declaration, or their strengths are equal.

Wor and trior nets:

These net types are *wired or* nets. It means that if one of their drivers has a value of 1, then the result value will also be 1.

Table : Truth table for wor and trior nets

	0	1	x	z
0	0	1	x	0
1	1	1	1	1
x	x	1	x	x
z	0	1	x	z

This table applies only to **wor** and **trior** nets driven by multiple drivers that have no strength declaration, or their strengths are equal.

Tri0 and tri1 nets:

These nets are used to model resistive pulldown and pullup devices. If a **tri0** net has no driver its value is 0. If a **tri1** net has no driver, then its value is 1. These values have pull strength.

Table: Truth table for tri0 net

	0	1	x	z
0	0	x	x	0
1	x	1	x	1
x	x	x	x	x
z	0	1	x	0

Table : Truth table for tri1 nets

	0	1	x	z
0	0	x	x	0
1	x	1	x	1
x	x	x	x	x
z	0	1	x	1

Supply0 and supply1 nets:

These nets are used to model power supplies in the circuit. They should have supply strength.

Trireg nets:

The **trireg** nets are used to model net capacity. If the **trireg** net driver has 0, 1, or x value, then it becomes a **trireg** net value and its strength can be one of the drive strengths, depending on the driver strength. If the driver has a z value, then the **trireg** net keeps the previously driven value and the strength can be one of the charge strengths, depending on the strength specified during net declaration.

reg
integer
time
real
realtime

Register Data Types

Registers are data types that store assigned values until a new assignment occurs. A new value can be assigned to registers only by using procedural assignments.

The **reg** register is a 1-bit wide data type. If more than one bit is required then range declaration should be used. Reg data type variables can be declared as memory.

The **integer** register is a 32-bit wide data type. Integer declarations cannot contain range specification. Integer variables can be declared as memory. Integers store signed values.

The **time** register is a 64-bit wide data type that is used to store simulation time and to check timing dependence. Time type registers store values as unsigned numbers. Time declaration cannot contain range specification. Time data type variables can be declared as memories.

The **real** register is a 64-bit wide data type that stores floating-point values. Real registers cannot be used with event control, concatenations ({}), modulus operator (%), case equality (===, !==), bit-wise operators (~, &, |, ^, ^~, ~^), reduction operators (^, ~^, ^~, &, &~, |, |~) and shift operators (<<, >>). Bit-selects and part-selects on real type variables are not allowed.

The **realtime** registers are treated in the same way as real registers.

64-bit

Examples

regscal;
reg [7:0] vect;
reg [7:0] mem [31:0];
integeri;
integeri_mem [7:0];
time t;
timet_mem [3:0];
real r;
realtime rt1, rt2;

'scal' is a 1-bit wide **reg** type register.

'vect' is an 8-bit wide **reg** type register.

'mem' is a **reg** type memory of 32 8-bit words.

'i' is an **integer** type register.

'i_mem' is an **integer** type memory.

't' is a **time** type register.

't_mem' is a **time** type memory.

'r' is a **real** type register.

'rt1' and 'rt2' are **realtime** type registers.

Important Notes

- New values can be assigned to registers only by using procedural statements.
- Some operators, bit-selects, part-selects, range and memory declaration are cannot be used with **real** and **realtime** type registers.
- Range can be specified only for **reg** data type registers.
- **Real** and **realtime** registers are initialized to zero value. **Reg**, **integer** and **time** registers are initialized to unknown value (x).

Vectors

Nets or reg data types can be declared as vectors (multiple bit widths). If bit width is not specified, the default is scalar (1-bit)

Vectors can be declared at [high# : low#] or [low# : high#], but the left number in the squared brackets is always the most significant bit of the vector.

Vector range specification contains two constant expressions: the msb (most significant bit) constant expression, which is the left-hand value of the range and the lsb (least significant bit)

constant expression, which is the right-hand value of the range. The msb and lsb constant expressions should be separated by a colon.

Both the msb constant expression and the lsb constant expression can be any value - positive, negative, or zero. The lsb constant expression can be greater, equal or less than the msb constant expression.

Vectors can be declared for all types of **net** data types and for **reg** data types. Specifying vectors for **integer**, **real**, **realtime**, and **time** data types is illegal.

Vector nets and registers are treated as unsigned values.

Examples

Example 1

reg [3:0] addr;

The 'addr' variable is a 4-bit vector register made up of addr[3] (the most significant bit), addr[2], addr[1], and addr[0] (the least significant bit).

Example 2

wire [-3:4] d;

The d variable is 8-bit vector net made up of d[-3] (msb), d[-2], d[-1], d[0], d[1], d[2], d[3], d[4] (lsb).

Example 3

tri [5:0] x, y, z;

The above line declares three 6-bit vectors.

Vector Part Select

For the vector declarations, it is possible to address bits or parts of vectors.

```
wire [31:0] busA,busB,busC; // 3 buses of 32-bit width.
busA[7] // bit # 7 of vector busA
bus[2:0] // Three least significant bits of vector bus, using bus[0:2] is illegal because the
                    //  significant bit should

reg [0:40] virtual_addr; // Vector register, virtual address 41 bitswide
virtual_addr[0:1] // Two most significant bits of vector virtual_addr
```

Variable Vector Part Select

Another ability provided in Verilog HDl is to have variable part selects of a vector. This allows part selects to be put in for loops to select various parts of the vector.

There aretwo special part-select operators:

[<starting_bit>+:width] - part-select increments from starting bit
[<starting_bit>-:width] - part-select decrements from starting bit

The starting bit of the part select can be varied, but the width has to be constant.

Thefollowing example shows the use of variable vector part select:

Examples

```
reg [255:0] data1;
reg [0:255] data2;
reg [7:0] byte;
//Using a variable part select, one can choose parts
byte = data1[31-:8]; //starting bit = 31, width =8 => data[31:24]
byte = data1[24+:8]; //starting bit = 24, width =8 => data[31:24]
byte = data2[31-:8]; //starting bit = 31, width =8 => data[24:31]
byte = data2[24+:8]; //starting bit = 24, width =8 => data[24:31]
```

Important Notes

- Both the msb and the lsb expressions should be constant expressions.
- The msb and the lsb constant expressions may be positive, negative, or zero.
- The lsb constant expression may be greater, equal or less than the msb constant expression.
- Vectors can be declared only for **nets** and **reg** data types.
- Vector declaration for **integer**, **real**, **realtime,** and **time** data types are illegal.

Verilog Operators

Operators act on the operands to produce desired results. Verilog provides various types of operators are

List of operators:

Arithmetic:	+ -- * /
Modulus:	%
Relational:	< <= > >=
Logical:	! && ||
Logical equality:	== !=
Case equality:	=== !==
Bit-wise:	~ & | ^ ~^ ^~
Reduction:	& ~& | ~| ^ ~^ ^~
Shift:	<< >>
Conditional:	? :
Event or:	or
Concatenations:	{} {{}}

Priority: This list provides priority of Verilog Operators:

Operator	Type
+ - ! ~	Unary
* / %	Arithmetic
+ - (binary)	Binary
<<>>	Shift
<<= > =>	Relational
== != === !==	Equality
& ~&	and nand
^ ~^ ^~	xorxnor
| ~|	or nor
&&	Logical and
||	Logical or

?:	Conditional operator

1. **Arithmetic operators:** This table gives description of arithmetic operators.

Operator	Description
a + b	(Addition) a **plus** b
a - b	(Subtraction) a **minus** b
a * b	(Multiplication) a **multiplied by** b
a / b	(Division) a **divide by** b
a **%** b	(Modulus) a **modulo** b

The modulus operator is not allowed for real data type variables. Other arithmetic operators can be used with all data types. For the modulus operator, the result takes the sign of the first operand. This code explains the usage of some basic operators:

```
module operators();
reg [3:0] a, b;
initial
begin
    a = 3;
    b = 5;
    $display(a + b );    //  8
    $display(a * b );    //  15
    $display(a / b );    //  0
    $display(b / a );    //  1
    $display(a % b );    //  3
    $display(7 % 3 );    //  1
    $display(7 % -3 );   //  1
    $display(-7 % 3 );   // -1
    $display(-7 % -3 );  // -1
end
endmodule
```

Output:

```
8
15
0
1
3
1
1
-1
-1
```

2. Relational operators

The relational operators are used to compare expressions. The value returned by the relational operators is 0 if the expression evaluates to false and 1 if expression evaluates to true. Following table gives the list of relational operators.

Operator	Description
a < b	a **less than** b
a > b	a **greater than** b
a <= b	a **less than or equal to** b
a => b	a **greater than or equal to** b

3. Equality operators

The equality operators are used to compare expressions. If a comparison fails, then the result will be 0, otherwise it will be 1.

If both operands of logical equality (==) or logical inequality (!=) contain unknown (x) or a high-impedance (z) value, then the result of comparison will be unknown (x). Otherwise it will be 1 or 0.

If operands of case equality (===) or case inequality (!==) contain unknown (x) or a high-impedance (z) value, then the result will be calculated bit by bit.

4. Logical operators

The logical operators are used to connect expressions.

Operator	Description
a && b	a **and** b
a || b	a **or** b
!a	**not** a

The result for these operators is 0 (when false), 1 (when true), and unknown (x - when ambiguous). The negation operator (!) turns a nonzero or true value of the operand into 0, zero or false value into 1, and ambiguous value of operator results in x (unknown value).

5. Bit-wise operators

The bit-wise operators calculate each bit of results by evaluating the logical expression on a pair of corresponding operand bits.

&	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

|	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

^	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

~^ ^~	0	1	x	z
0	1	0	x	x
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

~	**Result**
0	1
1	0
x	x
z	x

6. Shift operators

The shift operators perform left and right shifts on their left operand by the number of positions specified by their right operand. All vacated bits are filled with zeroes. If the expression that specifies the number of bits to shift (right operand) has unknown (x) or high-impedance (z) value, then result will be unknown.

Example:

```
module example();

reg[3:0] a, b;
reg[7:0] c, d;

initial
begin
   a =4'b1110;//14
```

```
  b =4'b0110;//5
$display("%b", a < b );// false - 0
$display("%b", a >8);// true - 1
$display("%b", a <= b );// false - 0
$display("%b", a >=10);// true - 1
$display("%b", a <4'b1zzz);// unknown - x
$display("%b", b <4'b1x01);// unknown - x
  a =4'b1100;
  b =4'b101x;
$display("%b", a ==4'b1100);// true - 1
$display("%b", a !=4'b1100);// false - 0
$display("%b", a ==4'b1z10);// false - 0
$display("%b", a !=4'b100x);// true ? 1
$display("%b", b ==4'b101x);// unknown - x
$display("%b", b !=4'b101x);// unknown - x
$display("%b", b ===4'b101x);// true - 1
$display("%b", b !==4'b101x);// false - 0

  a =4'b1100;
  b =4'b0000;
$display("%b",!a );// 0 - false
$display("%b",!b );// 1 - true
$display("%b", a && b );// 0 - false
$display("%b", a || b );// 1 ? true

  c =8'b1010xzxz;
  d =8'b10010011;
$display("%b", c & d );//= 8'b100000xx;
$display("%b", c | d );//= 8'b1011xx11;
$display("%b", c ^ d );//= 8'b0011xxxx;
$display("%b", c ~^ d );//= 8'b1100xxxx;
$display("%b",~ c );//= 8'b0101xxxx;

  a =4'b1111;
$display("%b", a <<3);//= 4'b1000
$display("%b", a >>3);//= 4'b0001
$display("%b", a <<1'bz);//= 4'bxxxx
$display("%b", a >>1'bx);//= 4'bxxxx
end

endmodule
```

7. Reduction operators

The reduction operator produces a 1-bit result. This result is calculated by recursively applying bit-wise operation on all bits of the operand. At each step of this recursive calculation the logical bit-wise operation is performed on the result of a previous operation and on the next bit of the operand. The operation is repeated for all bits of the operand.

value	&	~&	|	~|	^	~^	^~
4'b0000	0	1	0	1	0	1	1
4'b0001	0	1	1	0	1	0	0
4'b0011	0	1	1	0	0	1	1
4'b0111	0	1	1	0	1	0	0
4'b1111	1	0	1	0	0	1	1
4'b01xx	0	1	1	0	x	x	x
4'b01z0	0	1	1	0	x	x	x

Reduction operators

8. Concatenations Operator

The concatenation operator ({, }) provides a mechanism to append multiple operands.

Concatenations are expressed as operands within braces, with commas separating the operands.

Operands can be scalar nets or registers, vector nets or registers, bit-select, part-select, or sized constants.

```
// A = 1'b1, B = 2'b00, C = 2'b10, D = 3'b110
Y = {B , C}                    // Result Y is 4'b0010
Y = {A , B , C , D , 3'b001} // Result Y is 11'b10010110001

Y = {A , B[0], C[1]}          // Result Y is 3'b101
```

9. Replication Operator

Repetitive concatenation of the same number can be expressed by using a replication constant.
A replication constant specifies how many times to replicate the number inside the brackets ({ }).

```
reg A;
reg [1:0] B, C;
reg [2:0] D;
A = 1'b1; B = 2'b00; C = 2'b10; D = 3'b110;
Y = { 4{A} }                // Result Y is 4'b1111
Y = { 4{A} , 2{B} }        // Result Y is 8'b11110000
Y = { 4{A} , 2{B} , C }    // Result Y is 8'b1111000010
```

10. Conditional Operator

The conditional operator(?:) takes three operands.

Usage: condition_expr ?true_expr :false_expr ;

Conditional operators are frequently used in dataflow modeling to model conditional assignments. The conditional expression acts as a switching control.

```
//model functionality of a tristate buffer
assignaddr_bus = drive_enable ? addr_out : 36'bz;
//model functionality of a 2-to-1 mux
assign out = control ? in1 : in0;
```

Modeling Styles:

Data-Flow Modeling

Boolean Equation Based Models of Combinational Logic:

A Boolean equation describes combinational logic by an expression of operations on variables. Its counterpart in Verilog is the **continuous assignment** statement.

Considering a circuit as shown in Fig 1.1 which has 5 input And Or Invert (AOI) logic. This can be described using continuous assignment statement.

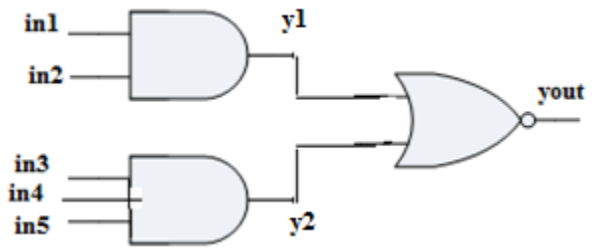


Fig 1.1 AND –OR-Invert Logic

Using DataFlow Modeling Fig 2.1 can be described using verilogis :

```
moduleAOIlogic ( in1, in2, in3, in4, in5, yout);
input  in1, in2, in3, in4, in5;
outputyout;
assignyout = ~((in1 & in2)| ( in3 & in4 &in5));
endmodule
```

The keyword assign declares a continuous assignment and binds the Boolean expression on RHS of the statement to the variable on LHS. The above verilog program uses bitwise operators such as ~ for inverter, & for AND , and | for OR operation. Any time a variable in RHS changes during simulation, the RHS expression reevaluated and the result is used to update the LHS.

The Fig 1.1 is modified to have an additional input, enable as shown Fig 1.2.

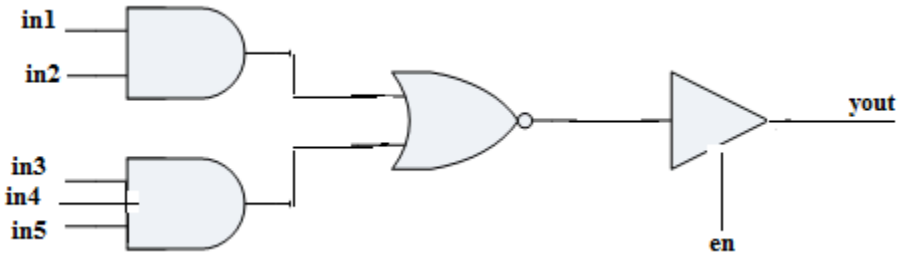


Fig 1.2 Modified And –Or- Invert Logic with enable.

The functionality of Fig 1.2 is described using **conditional** operator.

```
moduleAOI_modified_logic ( in1, in2, in3, in4, in5,en,yout);
input  in1, in2, in3, in4, in5,en;
outputyout;
assignyout = en?~((in1 & in2)| ( in3 & in4 &in5)) : 1'bZ;
endmodule
```

The continuous assignment operator (?:) acts like a if then else statement that selects between two expression. In Fig 1.2, when en = 1 (true) it evaluates the expression which is right hand side of ?. When en = 0 (false) it evaluates the expression after : . This example also illustrates how to write models that include 3 state outputs. The value of yout is formed by combinational logic while enable is asserted, but has the value Z otherwise.
A module may contain multiple continuous assignments, and they are active concurrently with all other continuous assignments, primitives, behavioral statements, and instantiated modules, just as all of the electrical signals in a circuit are active concurrently.
The model described by AOI_modified_logic declares yout to be of type **wire**, and associates with it a Boolean expression that defines the value of yout.

```
moduleAOI_modified_logic ( in1, in2, in3, in4, in5,en,yout);
input  in1, in2, in3, in4, in5,en;
outputyout;
wireyout = en?~((in1 & in2)| ( in3 & in4 &in5)) : 1'bZ;
endmodule
```

Acontinuous assignment statement with conditional operator provides a convenient way to model the multiplexer circuit shown in Fig 1.3.

```
moduleMux2_32 ( data1, data0, select, muxout);
parameterword_size = 32;
input [word_size-1:0] data1, data0;
input select
output [word_size-1:0] muxout;
assignmuxout = select ?data1 : data0;
endmodule
```

Note that the size of the 32-bit datapath in Mux2_32 is specified by a **parameter** to make the model scalable, portable, and more useful. Parameter are constant values, & remain fixed during simulation.

1.2 Propagation Delay and Continuous Assignments:

Propagation delay can be associated with a continuous assignment so that its implicit logic has the same functionality and timing characteristics as its gate level counterpart. The functionality of an AOI gate structure with unit propagation delays on its implicit gates is described below by AOI_delay_logic. (Refer Fig 1.1)

```
moduleAOI_delay_logic(in1, in2, in3, in4, yout);
inputin1, in2, in3, in4;
outputyout;
wire #1 y1 = in1 & in2;
wire #1 y2= in3 & in4;
wire #1 yout = ~( y1|y2);
endmodule
```

Each declaration of a wire includes a logic expression assigning value to the wire, and includes a unit time delay. The three continuous assignments are active concurrently during simulation, each having a monitoring mechanism, implemented by the simulator, that detects a change in its RHS expression and schedules a change to update the LHS variable.